

BÁO CÁO TỔNG KẾT TRAINING DATA ENGINEER

Xây dựng hệ thống phát hiện xâm nhập thời gian thực

Streaming IDS · Apache Kafka + Spark Structured Streaming

Lê Anh Quân

Người hướng dẫn: Nguyễn Công Thịnh

28 · 08 · 2026

NỘI DUNG

01	Bài toán	Những câu hỏi cần giải quyết là gì?	2 phút
02	Hệ thống	Kiến trúc và quyết định thiết kế	4 phút
03	Demo trực tiếp	Hệ thống đang được cho chạy nền	5 phút
04	Kết quả đo	Độ chính xác, và điểm yếu mà phép đo tìm ra	2 phút
05	Giới hạn và sự cố	Giới hạn/sự cố nào có thể được vượt qua	5 phút
06	Kết luận	Những hạn chế còn sót lại và hướng phát triển	2 phút

Bài toán

Xây một pipeline thời gian thực đánh giá từng bản ghi theo một tập luật và đo được chính nó.

01

Thông lượng

Xử lý kịp tốc độ dữ liệu đến,
không tích lũy độ trễ vô hạn

02

Độ trễ

Khoảng cách từ lúc bản ghi sinh
ra tới lúc có cảnh báo

03

Độ chính xác đo được

Tính từ dữ liệu đã lưu, không
phải ước lượng

04

Chi phí lưu trữ

Không thể lưu toàn bộ lưu lượng
mạng về lâu dài

Bốn trục này sẽ được đo lại ở cuối bài.

Phạm vi và lựa chọn phương pháp

Rule-based, không dùng Machine Learning

Trọng tâm đề án là hạ tầng dữ liệu. Rule-based giữ được khả năng giải thích - mỗi cảnh báo luôn chỉ ra được luật nào đã kích hoạt.

Sáu nhóm hành vi tấn công

- DDoS
- Brute Force
- Port Scanning
- Malware Phoning Home (C2)
- Data Exfiltration
- Malicious File Download

Một máy tính cá nhân

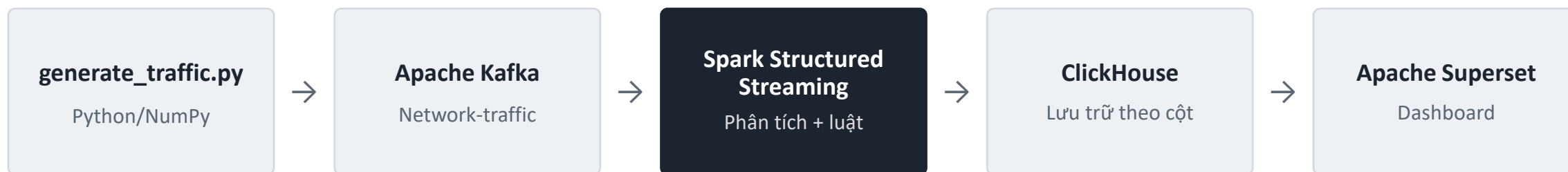
Không phải cụm phân tán. Dữ liệu mô phỏng theo lược đồ NSL-KDD, không phải lưu lượng thật.

ĐIỂM CẦN NHẮM TRƯỚC

Một dòng NSL-KDD là một kết nối đã được tóm tắt — không phải một gói tin, cũng không phải một cuộc tấn công.

Trường count mang nghĩa “số kết nối tới cùng một host trong 2 giây vừa qua”. Chính vì mỗi dòng đã mang sẵn ngữ cảnh cửa sổ nên các luật ngưỡng có thể hoạt động trên từng dòng độc lập, không cần trạng thái.

Kiến trúc



Vì sao lại chọn Kafka

Tách rời bộ sinh và Spark. Dữ liệu được lưu theo offset nên có thể phát lại toàn bộ luồng - đã dùng nhiều lần để đánh giá lại trên đúng cùng một tập dữ liệu.

Vì sao lại chọn Spark

Cùng một API DataFrame cho cả xử lý luồng và xử lý lô. Nhờ đó bộ máy luật được chấm điểm ngoại tuyến trong 30 giây mà không phải viết lại dòng nào.

Vì sao lại chọn ClickHouse

Dữ liệu cần lưu có dạng nhật ký chỉ ghi thêm, truy vấn chủ yếu là tổng hợp trên vài cột. Lưu trữ theo cột đọc đúng cột cần thiết thay vì cả 43 cột mỗi dòng.

Luật không được phép nhìn thấy đáp án

```
FORBIDDEN_FIELDS = {"label"}
ALLOWED_FIELDS   = set(_SCHEMA_TYPES) - FORBIDDEN_FIELDS

if field in FORBIDDEN_FIELDS:
    raise RuleLoadError(
        f"{where}: field '{field}' is not "
        "available to the detection logic."
    )
```

Một IDS thật không bao giờ biết trước sự thật. Nếu luật được nhận, độ chính xác sẽ đạt 100% một cách hoàn toàn vô nghĩa.

Hệ thống từ chối khởi động nếu bất kỳ luật nào tham chiếu tới label.

Nhãn chỉ được dùng sau khi phát hiện xong để chấm điểm.

Thiết kế lưu trữ hai bảng

`detections`

5.56%

của toàn bộ lưu lượng đi vào

43 cột đầy đủ, nhưng chỉ với những bản ghi đáng lưu: mọi tấn công thật và mọi bản ghi bị gán cờ. Lưu lượng bình thường được bỏ qua.

`traffic_counts`

4 cột

vài dòng mỗi micro-batch

`batch_id · batch_time · label · record_count`. Chỉ lưu số đếm, không lưu bản ghi -> chi phí gần như bằng không.

Tỷ lệ báo động giả (False Positive Rate - FPR) cần mẫu số là toàn bộ lưu lượng bình thường - thứ mà hệ thống cố tình không lưu. Bảng thứ hai cấp đúng mẫu số đó.

Thiết kế lưu trữ xuất phát từ câu hỏi cần trả lời, không phải từ dữ liệu sẵn có.

DEMO

Hệ thống đã chạy từ đầu buổi · 2,500 bản ghi/giây

01

Các ô chỉ số đang tăng thật

không phải ảnh chụp

02

Bảng cảnh báo — đọc to một dòng

gom theo cửa sổ phút, không phải từng bản ghi

03

Một dòng FALSE POSITIVE

hầu hết dashboard IDS giấu cột này

04

Biểu đồ độ trễ, đường tham chiếu 2 giây

khoảng cách này là nội dung slide kế tiếp

Kết quả đo

Recall

98.5%

bắt được 98.5% tấn công thật

Precision

89.9%

89.9% số cảnh báo là đúng

F1-score

94.0%

trung bình điều hòa

Tỷ lệ báo động giả (FPR)

0.584%

trên lưu lượng bình thường

Accuracy = 99.37% - nhưng ngưỡng cơ sở là 95.0%

Lưu lượng bình thường chiếm tới 95% tổng số lượng bản ghi, nên một hệ thống không phát hiện gì cả vẫn đạt Accuracy = 95%. Con số 99.37% chỉ hơn ngưỡng đó 4.37 điểm. Recall và tỷ lệ báo động giả không thể bị đánh lừa bằng cách không làm gì, nên chúng là căn cứ đánh giá chính.

Tái lập qua ba lần chạy độc lập, chênh nhau hai bậc về quy mô (695,000 · 1,595,000 · 1,610,000 bản ghi) — sai lệch dưới 0.1 điểm phần trăm.

Điểm yếu mà phép đo tìm ra

Loại tấn công	Recall	Precision
brute_force	100%	100%
ddos	100%	100%
port_scan	100%	100%
malware_c2	100%	95.8%
data_exfiltration	96.6%	93.0%
malicious_download	94.3%	62.7%

Chỉ lộ ra nhờ tính precision theo từng loại tấn công.

Một bản ghi bị gán nhầm sang loại tấn công khác vẫn bị tính là sai cho loại đó. Nếu chỉ dùng chỉ số nhị phân “có phát hiện hay không”, điểm yếu này bị che khuất hoàn toàn.

Đây không phải lỗi cần sửa cho bằng 100%. Hai luật cuối là luật ngưỡng theo dung lượng, nằm ngay trên ranh giới phân phối của lưu lượng bình thường — hạ ngưỡng thì Recall tăng nhưng Precision sập.

Chi phí một micro-batch

$$\text{batch_ms} = 1,618 + 0.0405 \times N$$

Kích thước lô	Thời gian	Trên mỗi bản ghi
0 bản ghi	19 ms	—
4,500	1,800 ms	400 μ s
10,700	2,000 ms	187 μ s
15,000	2,225 ms	148 μ s

73%

thời gian của một lô 15,000 bản ghi là chi phí cố định – không phải công việc thực trên dữ liệu.

Khối lượng công việc tăng 3.3 lần nhưng thời gian chỉ tăng 1.24 lần. Vì độ trễ xấp xỉ 1.5 lần thời gian một lô, tồn tại một sàn khoảng 2.5 giây mà không kích thước lô nào vượt qua được.

Mốc 2 giây thuộc về tầng kiến trúc nào?

Tầng cảm biến nội tuyến

Snort 3 · Suricata — chạy trực tiếp trên đường truyền

350 μ s – vài ms

Tầng phân tích luồng

Thiết kế tham chiếu IDS trên ksqldb do chính Confluent công bố

cửa sổ 60 giây

Mục tiêu của đề án

Không thuộc về tầng nào trong hai tầng trên

2 giây

Chậm hơn khoảng 5,700 lần so với một cảm biến thật, và nhanh hơn khoảng 30 lần so với thiết kế IDS do chính công ty tạo ra Kafka công bố. Con số 2 giây không đến từ truyền thống nào cả — nó rơi vào khoảng trống giữa hai kiến trúc.

Sai lệch nằm ở yêu cầu, không nằm ở sản phẩm.

Sự cố kỹ thuật đáng kể nhất

- | | | |
|----|--------------------|--|
| 01 | Hiện tượng | isActive = true, trạng thái “Processing new data”, hơn 80 giây. Không lỗi, không cảnh báo, không dữ liệu. |
| 02 | Loại trừ | Viết một công cụ chẩn đoán riêng: Kafka đọc được 1,372,349 bản tin trong 10 giây — hoàn toàn khỏe mạnh. |
| 03 | Thread dump | Luồng ở trạng thái RUNNABLE trong Project.getAllValidConstraints — đang tiêu tốn CPU, không phải đang chờ. |
| 04 | Nguyên nhân | Cây biểu thức chứa 119 lời gọi coalesce. Catalyst so sánh cấu trúc theo từng cặp, chi phí tăng theo $O(n^2)$, và lặp lại ở mỗi micro-batch. |
| 05 | Khắc phục | Tắt constraintPropagation, đồng thời giảm 45% số node bằng element_at. Thời gian ngay lập tức giảm từ vô định xuống còn 6 mili-giây. |

Một lỗi im lặng khó tìm hơn một lỗi to tiếng. Sự cố tràn bộ nhớ ở mục V.3 ném ra stack trace và được sửa trong vài phút; lỗi này không để lại dấu vết nào.

Các vấn đề khác đã xử lý

Bộ đếm bản ghi lỗi luôn bằng 0

from_json ở chế độ PERMISSIVE trả về struct không null với mọi trường bên trong là null

Tràn bộ nhớ do micro-batch không giới hạn

Thiếu maxOffsetsPerTrigger; toàn bộ tồn đọng gom thành một lô rồi thu về driver

ClickHouse 25.1 chặn truy cập mạng

Người dùng default không mật khẩu bị vô hiệu hóa; healthcheck vẫn báo khỏe mạnh

Chuỗi ràng buộc phiên bản

Một dòng phụ thuộc không ghim phiên bản kéo theo ba lựa chọn sai ở ba tầng khác nhau

Môi trường Spark trên Windows

winutils, HADOOP_HOME, Python 3.11, connector Kafka nạp qua --packages

Sẵn sàng đi sâu vào bất kỳ mục nào ở phần hỏi đáp.

Hạn chế

Không đạt mục tiêu độ trễ 2 giây

Đo được ~3.3 giây. Là giới hạn kiến trúc, không phải lỗi triển khai (đã được phân tích ở trên).

Không thể tương quan theo kẻ tấn công

NSL-KDD cố tình không chứa trường định danh nào -> hệ thống không trả lời được “địa chỉ nào đã quét cổng rồi tải tệp độc hại”.

Bộ sinh không mô hình hóa tấn công như một phiên

Mỗi bản ghi sinh độc lập, nên một đợt tấn công brute force xuất hiện dưới dạng hàng nghìn sự kiện rời rạc thay vì một sự kiện có diễn biến.

Một máy, dữ liệu mô phỏng

Số đo phản ánh hạn chế của cấu hình đó. Không thể ngoại suy từ dữ liệu mô phỏng sang lưu lượng mạng thật.

Hướng phát triển

Chuyển việc phát hiện sang biên

Đưa sáu luật vào chính bộ sinh dưới dạng phép toán mảng boolean — tốn thêm vài micro-giây mỗi bản ghi. Kafka mang theo sẵn kết luận phát hiện; Spark làm đúng việc của một hệ micro-batch: lưu trữ, tổng hợp, dashboard. Độ trễ phát hiện giảm từ 3.3 giây xuống còn vài micro-giây. Đây chính xác là cách Snort và Suricata hoạt động.

Viết lại sink ClickHouse

Gộp hai lệnh ghi HTTP, ghi từ executor thay vì thu về driver. Ước tính đưa độ trễ về 2.1 - 2.6 giây.

Bổ sung trường định danh

Mở khóa khả năng tương quan theo kẻ tấn công — đánh đổi là không còn tuân thủ chuẩn NSL-KDD.

Mô hình hóa tấn công như một phiên

Số kết nối tăng dần theo thời gian trong một phiên thay vì giá trị ngẫu nhiên độc lập.

KẾT LUẬN

Đo được giới hạn của hệ thống mình xây, và giải thích được nguồn gốc của giới hạn đó, có giá trị không kém việc đạt một con số đặt ra từ trước khi có bất kỳ phép đo nào.

1,610,000

bản ghi trong một lần chạy

98.5%

Recall, tái lập qua ba lần chạy

6,700

bản ghi/giây, đầu tới cuối

~3.3 s

độ trễ phát hiện trung bình

Cảm ơn vì đã lắng nghe

Hỏi đáp